

```

> restart :
> with(LinearAlgebra) :
  with(Optimization) :
  with(GraphTheory) :
> #Number of firms in the network
  N := 6

```

(1)

```

> Q01 := 50 :
  Q02 := 1 :
  Q03 := 25 :
  Q04 := 1 :
  Q05 := 10 :
  Q06 := 1 :
  Q07 := 5 :
  Q08 := 1 :
> Q0 := Vector(N) :
  for i from 1 to N do
    Q0[i] := eval(parse(sprintf('Q0%d', i))) :
  end do:
  Q0

```

$$\begin{bmatrix} 50 \\ 1 \\ 25 \\ 1 \\ 10 \\ 1 \end{bmatrix}$$

(2)

```

> Dc := 1

```

(3)

```

> Ec := 1

```

(4)

```

> Ms := 100

```

(5)

```

> L12 := 1 :
  L13 := 0.75 :
  L14 := 0.75 :
  L15 := 0.75 :
  L16 := 0.75 :
  L17 := 0.75 :
  L18 := 0.75 :
  L23 := 0.75 :

```

```

L24 := 0.75 :
L25 := 0.75 :
L26 := 0.75 :
L27 := 0.75 :
L28 := 0.75 :
L34 := 1 :
L35 := 0.75 :
L36 := 0.75 :
L37 := 0.75 :
L38 := 0.75 :
L45 := 0.75 :
L46 := 0.75 :
L47 := 0.75 :
L48 := 0.75 :
L56 := 1 :
L57 := 0.5 :
L58 := 0.5 :
L67 := 0.5 :
L68 := 0.5 :
L78 := 1 :

#Define Link matrix
L := Matrix(N, N, 1) :
for i from 1 to N do
  L[i, i] := 1 :
  for j from i + 1 to N do
    Lij := parse(sprintf('L%d%d', i, j));
    if type(Lij, nonnegative) then
      L[i, j] := Lij;
      L[j, i] := Lij;
    else
      L[i, j] := 0 :
      L[j, i] := 0 :
    end if;
  end do;
end do;
L;

```

$$\begin{bmatrix}
1 & 1 & 0.75 & 0.75 & 0.75 & 0.75 \\
1 & 1 & 0.75 & 0.75 & 0.75 & 0.75 \\
0.75 & 0.75 & 1 & 1 & 0.75 & 0.75 \\
0.75 & 0.75 & 1 & 1 & 0.75 & 0.75 \\
0.75 & 0.75 & 0.75 & 0.75 & 1 & 1 \\
0.75 & 0.75 & 0.75 & 0.75 & 1 & 1
\end{bmatrix}$$

(6)

```

> firmNames := [seq(sprintf("F%dQ%d", i, Q0[i]), i = 1 ..N) ] :
G := Graph(N, weighted) :
  for i from 1 to N do
    for j from i + 1 to N do
      if L[i,j] ≠ 0 then
        AddEdge(G, {i,j} ) :
        SetEdgeWeight(G, {i,j} , L[i,j] ) :
      end if:
    end do:
  end do:

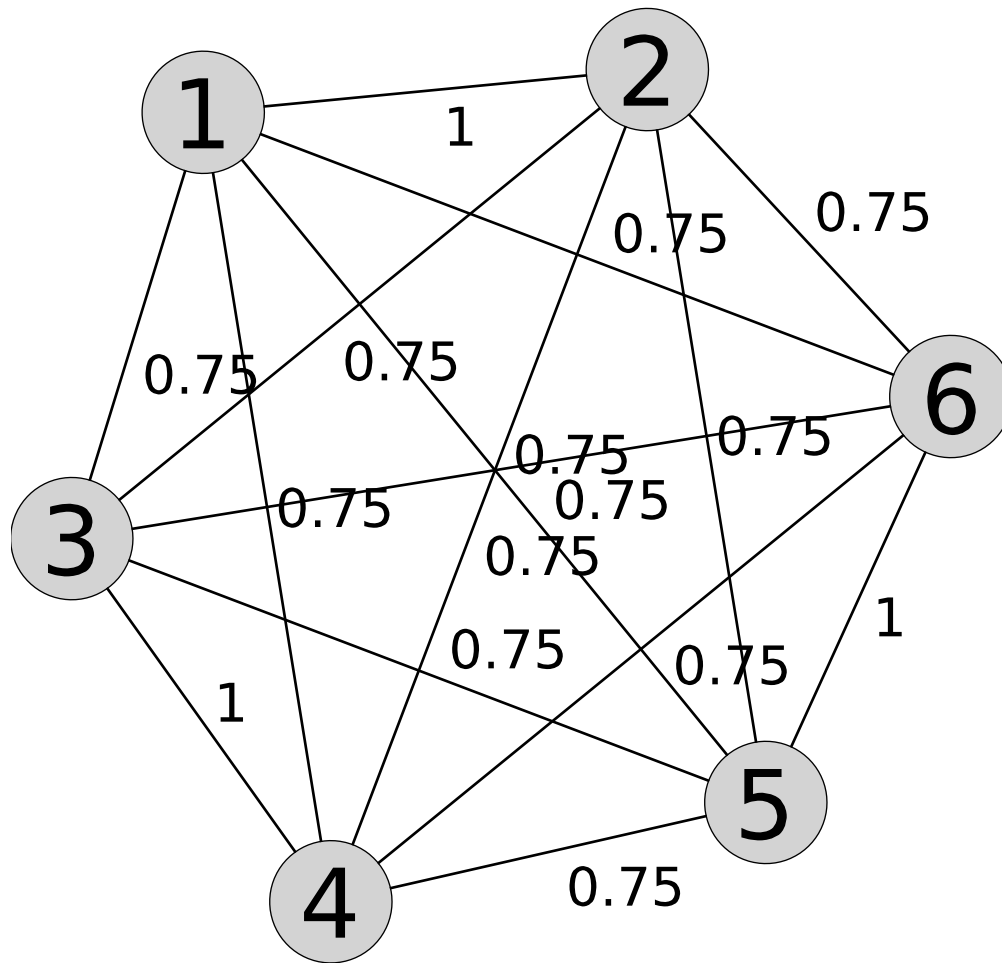
SetVertexLabels(G, firmNames) :
StyleVertex(G, Vertices(G), shape = "circle", font = ["Helvetica", 36]) :

for edge_link in Edges(G) do
  weight_val := GetEdgeWeight(G, edge_link) :
  SetEdgeAttribute(G, edge_link, label = convert(weight_val, string) ) :
end do:

StyleEdge(G, Edges(G), font = ["Helvetica", 20]) :
GraphTheoryOptions(vertexsize = 3) :
DrawGraph(G, style = spring);

GraphTheoryOptions(vertexsize = default) :

```



```

> Cd := Vector(N) :
  for i from 1 to N do
    Cd[i] := Dc·xd[i]· $\left(1 + xe[i] + \frac{xd[i]}{2}\right)$  :
  end do:
Cd :

```

```

> xe := Vector(N) :
  for i from 1 to N do
    xe[i] := parse(sprintf('xe%d', i));
  end do:
xe :

```

```

> xd := Vector(N) :
  for i from 1 to N do
    xd[i] := parse(sprintf('xd%d', i));
  end do:
xd :

```

```

> Ce := Vector(N) :
  for i from 1 to N do
    Ce[i] := Vector[column](N);
    for j from 1 to N do
      if L[i,j] = 0 then

```

```
Ce[i][j] := piecewise(xe[i] = 0, 0, infinity);  
else
```

```
Ce[i][j] :=  $\frac{Ec \cdot xe[i]}{L[i,j]}$  ;
```

```
end if;
```

```
end do;
```

```
end do;
```

```
Ce :
```

```
> q := Vector(N) :
```

```
for i from 1 to N do
```

```
q[i] := Q0[i] + xd[i] + xe[i];
```

```
end do;
```

```
q :
```

```
> S := Vector(N) :
```

```
for i from 1 to N do
```

```
qsum := 0;
```

```
for j from 1 to N do
```

```
if j ≠ i then
```

```
qsum := qsum + L[i,j]·q[j];
```

```
end if;
```

```
end do;
```

```
S[i] := (Ms·q[i]2) / (q[i] + qsum) :
```

```
end do;
```

```
S :
```

```
>
```

```
> P := Vector(N) :
```

```
for i from 1 to N do
```

```
P[i] := Vector[column](N) :
```

```
for j from 1 to N do
```

```
P[i][j] := S[i] - Cd[i] - Ce[i][j] :
```

```
end do;
```

```
end do;
```

```
P :
```

```
> dPd := Vector(N) :
```

```
for i from 1 to N do:
```

```
dPd[i] := Vector[column](N) :
```

```
for j from 1 to N do
```

```
dPd[i][j] := diff(P[i][j], xd[i]);
```

```
end do;
```

```
end do;
```

```
dPd :
```

```
> dPe := Vector(N) :
```

```
for i from 1 to N do
```

```
dPe[i] := Vector[column](N) :
```

```
for j from 1 to N do
```

```

    dPe[i][j] := diff(P[i][j], xe[i]);
  end do:
end do:
dPe :

```

```

> xeMax := Vector(N) :
  for i from 1 to N do
    xeMax[i] := Vector[column](N) :
    for j from 1 to N do
      xeMax[i][j] := Q0[j] - Q0[i] :
    end do:
  end do:
  xeMax :

```

```

> #Iteration parameters
  maxIt := 100; #Maximum number of outer iterations
  tol := 1e-4; #Convergence tolerance
  df := 0.5; #Damping factor

                                maxIt := 100
                                tol := 0.0001
                                df := 0.5

```

```

> #Starting points
  iniTar := Vector(N, 1) :
  iniXe := Vector(N, 0) :
  iniXd := Vector(N, 0) :
  for i from 1 to N do
    maxXeMax := 0 :
    for j from 1 to N do
      if xeMax[i][j] ≥ maxXeMax then
        maxXeMax := copy(xeMax[i][j]) :
        iniTar[i] := copy(j) :
      end if:
    end do:
    iniXe[i] := copy(xeMax[i][iniTar[i]]) :
  end do:

```

```

> #First Iteration
  cXd := copy(iniXd) :
  cXe := copy(iniXe) :
  cTar := copy(iniTar) :

  nXd := Vector(N, datatype=float[8]) :
  nXe := Vector(N, datatype=float[8]) :
  nTar := Vector(N, datatype=integer) :

```

```

> #Iterated Best Response
  for c from 1 to maxIt do
    convergence := true :
    for i from 1 to N do

```

(7)

```

bXd[i] := cXd[i] :
bXe[i] := cXe[i] :
bTar[i] := cTar[i] :

#Substitution rules
subRules := { } :
for k from 1 to N do
  if k ≠ i then
    subRules := subRules union {xd[k] = cXd[k], xe[k] = cXe[k]} :
  end if:
end do:

#Set up
maxProf := -infinity :
for j from 1 to N do
  objFun := subs(subRules, P[i][j]) :
  cons := {xd[i] ≥ 0, xe[i] ≥ 0, xe[i] ≤ xeMax[i][j]} :

  #Update current values within constraints
  if xeMax[i][j] < 0 then
    iniXe[i] := 0 :
  else
    iniXe[i] := max(0, min(cXe[i], xeMax[i][j])) :
  end if:
  iniXd[i] := max(0, cXd[i]) :

  #Run optimization
  try
    result := Optimization[NLPSolve](
      objFun,
      cons,
      initialpoint = {xe[i] = iniXe[i], xd[i] = iniXd[i]},
      maximize = true,
      iterationlimit = 10000) :

  #Save results
  cProf := result[1] :
  solEq := result[2] :

  if cProf > maxProf then
    maxProf := cProf:
    bXd[i] := eval(xd[i], solEq) :
    bXe[i] := eval(xe[i], solEq) :
    bTar[i] := j :
  end if:
  catch:
    #printf("NLPSolve issue for firm %d, target %d. Error: %s\n", i, j, lastexception);
  end try:
end do: #End of j loop

```

```
 $nXd[i] := bXd[i] :$   
 $nXe[i] := bXe[i] :$   
 $nTar[i] := bTar[i] :$ 
```

```
end do: #End of  $i$  loop
```

```
 $convergence := true :$ 
```

```
for  $ic$  from 1 to  $N$  do
```

```
#Check convergence
```

```
if  $\text{abs}(nXd[ic] - cXd[ic]) > tol$  or  $\text{abs}(nXe[ic] - cXe[ic]) > tol$  or  $nTar[ic] \neq cTar[ic]$  then
```

```
 $convergence := false :$ 
```

```
break;
```

```
end if:
```

```
end do: #End of  $ic$  loop
```

```
if  $convergence$  then
```

```
 $\text{printf}(\text{"Converged to Nash Equilibrium after \%d iterations.\n"}, c) :$ 
```

```
 $cXd := \text{copy}(nXd);$ 
```

```
 $cXe := \text{copy}(nXe);$ 
```

```
 $cTar := \text{copy}(nTar);$ 
```

```
break;
```

```
end if:
```

```
for  $a$  from 1 to  $N$  do
```

```
 $cXd[a] := (1 - df) \cdot cXd[a] + df \cdot nXd[a];$ 
```

```
 $cXe[a] := (1 - df) \cdot cXe[a] + df \cdot nXe[a];$ 
```

```
end do:
```

```
 $cTar := \text{copy}(nTar) :$ 
```

```
if  $c = \text{maxIt}$  then
```

```
 $\text{printf}(\text{"Reached maximum of \%d iterations without convergence.\n"}, \text{maxIt}) :$ 
```

```
break;
```

```
end if:
```

```
 $\text{printf}(\text{"Iteration \%d: cXd = \%A, cXe = \%A, cTar = \%A\n"}, c, cXd, cXe, cTar);$  #DEBUG
```

```
end do: #End of  $c$  loop
```

Warning, no iterations performed as initial point satisfies first-order conditions

Warning, no iterations performed as initial point satisfies first-order conditions

Warning, no iterations performed as initial point satisfies first-order conditions

Warning, no iterations performed as initial point satisfies first-order conditions

Warning, no iterations performed as initial point satisfies first-order conditions

Warning, no iterations performed as initial point satisfies first-order conditions

Warning, no iterations performed as initial point satisfies first-order conditions

order conditions

Warning, no iterations performed as initial point satisfies first-order conditions

Warning, no iterations performed as initial point satisfies first-order conditions

Converged to Nash Equilibrium after 23 iterations.

```
> FinalProfit := Vector(N, datatype = float[8]) :
   FinalSales := Vector(N, datatype = float[8]) :
   fSubRules := { } :
   for i from 1 to N do
   fSubRules := fSubRules union {xd[i] = cXd[i], xe[i] = cXe[i]} :
   end do:

   printf("\nNash Equilibrium Result\n\n");
   printf("%-4s | %-3s | %-6s | %-6s | %-6s | %-7s | %-7s\n",
          "Firm", "Tar", "xe", "xd", "Q", "P", "Margin");
   printf("-----+-----+-----+-----+-----+-----+-----\n");

   for i from 1 to N do
   profitExpr := subs(fSubRules, P[i][cTar[i]]) :
   salesExpr := subs(fSubRules, S[i]) :

   FinalProfit[i] := evalf(profitExpr) :
   FinalSales[i] := evalf(salesExpr) :
   marginStr := sprintf("%.1f%%", evalf((FinalProfit[i] * 100) / (FinalSales[i]))) :

   printf("%-4d | %-3d | %-6.1f | %-6.1f | %-6.1f | %-7.2f | %-7s\n",
          i,
          # Firm
          cTar[i],
          # Chosen Target for reverse-engineering
          cXe[i],
          # Reverse-engineering Effort (xe)
          cXd[i],
          # Development Effort (xd)
          evalf(Q0[i] + cXd[i] + cXe[i]), # Final Quality (Q)
          FinalProfit[i],
          # Profit
          marginStr
          # Margin
          );

   printf("-----+-----+-----+-----+-----+-----+-----\n");
   end do:
```

Nash Equilibrium Result

Firm	Tar	xe	xd	Q	P	Margin
1	1	0.0	54.9	104.9	1962.36	55.7%
2	1	49.0	0.0	50.0	751.66	93.9%
3	1	25.0	9.8	59.8	851.98	71.7%

4		1		49.0		0.0		50.0		765.34		92.1%
5		1		40.0		0.0		50.0		784.14		93.6%
6		1		49.0		0.0		50.0		772.14		92.2%

>